

Side-Channel-Assisted Reverse-Engineering of Encrypted DNN Hardware Accelerator IP and Attack Surface Exploration

Cheng Gongye, Yukui Luo, Xiaolin Xu, and Yunsi Fei

Department of Electrical & Computer Engineering

Northeastern University, Boston, MA

Email: {gongye.c, luo.yuk, x.xu, y.fei}@northeastern.edu

Abstract—Deep Neural Networks (DNNs) have revolutionized numerous application domains with their unparalleled performance. As the models become larger and more complex, hardware DNN accelerators are increasingly popular. Field-Programmable Gate Array (FPGA)-based DNN accelerators offer near-Application Specific Integrated Circuit (ASIC) efficiency and exceptional flexibility, establishing them as one of the primary hardware platforms for rapidly evolving deep learning implementations, particularly on edge devices. This prominence renders them lucrative targets for attackers. Existing attacks aimed at compromising the confidentiality of DNN models deployed on FPGA DNN accelerators often assume complete knowledge of the accelerators. However, this assumption does not hold for real-world, proprietary, high-performance FPGA DNN accelerators. In this study, we introduce a comprehensive and effective reverse-engineering methodology for demystifying FPGA DNN accelerator soft Intellectual Property (IP) cores. We demonstrate its application on the cutting-edge AMD-Xilinx Deep Learning Processing Unit (DPU). Our method relies on schematic analysis and, innovatively, electromagnetic (EM) side-channel analysis to reveal the data flow and scheduling of the DNN accelerators. To the best of our knowledge, this research is the first successful endeavor to reverse-engineer a commercial encrypted DNN accelerator IP. Moreover, we investigate attack surfaces exposed by the reverse-engineering findings, including the successful recovery of DNN model architectures and extraction of model parameters. These outcomes pose a significant threat to real-world commercial FPGA-DNN acceleration systems. We discuss potential countermeasures and offer recommendations for FPGA-based IP protection.

1. Introduction

In the past decade, Deep Neural Networks (DNNs) have significantly advanced various application domains, such as classification, object detection, and natural language processing. Nevertheless, the complexity of DNN models demands acceleration to cope with the intensive computations. For example, EfficientNet-B3, one of the most efficient DNNs to date [1], requires 1.8 billion FLOPs for a single inference. Numerous DNN accelerators have been developed to expedite DNN operations, available on platforms like GPU, ASIC, and FPGA. While GPU and ASIC-based solutions, such as

TPU [2], have demonstrated success in training DNN models and inferring large data batches, FPGA-based accelerators exhibit unique advantages in DNN inference, particularly on edge devices.

Real-time applications like autonomous driving require low-latency, energy-efficient inference. In comparison to GPUs, FPGA-based DNN accelerators typically support fully quantized DNN models with specialized data paths, resulting in lower latency and power consumption. For instance, on the same technology node, an FPGA-based AMD-Xilinx DPU can run inferences with a ResNet-18 at 86.7 fps with the power consumption of 7.4 W [3], whereas a high-end NVIDIA Pascal Titan X incurs a power consumption of 250 W and achieves slightly better performance — 98.6 fps [4]. In contrast to ASIC, FPGA-based DNN accelerators offer greater flexibility, allowing adaptation to rapidly evolving deep learning developments and integration of the latest advancements.

Historically, FPGAs presented significant barriers for software developers and data scientists to deploy models efficiently. To address this issue, state-of-the-art FPGA-based accelerators now provide end-to-end toolchains for users, such as Vitis-AI and Apache TVM [5], [6]. These toolchains enable users to train a DNN model, compile it and execute the model on FPGA accelerators.

Several attacks have breached the confidentiality of FPGA DNN accelerators, recovering model architecture [7], [8], [9], parameters [10], [11], or input [12]. However, previous work has primarily targeted open-source or manually built DNN accelerators, requiring knowledge of the hardware structure and scheduling/data flow. In contrast, commercial closed-source FPGA DNN accelerators do not disclose such information. Taking the AMD-Xilinx DPU as an example, only configuration parameters are disclosed, specifying the accelerator's multi-dimension parallelism. Consequently, the crucial first step in studying the hardware security of encrypted DNN accelerator IP is to reverse-engineer both the hardware structure and scheduling/data flow, opening up new attack surfaces for further exploration.

We chose the AMD-Xilinx DPU as our reverse-engineering target because it is one of the most sophisticated to date. If the methodology is successful with DPU, it should be applicable to other simpler DNN accelerators. Table 1 compares DPU with relevant open-source projects,

TABLE 1. COMPARISON OF FPGA-BASED ACCELERATORS.

Project Year	Max Energy Efficiency (GOP/s/W)	Software Compatibility	Models	Supporting Tasks
DPU 2019	342	Pytorch Tensorflow CaffeV2	134 models [17]	Classification Object detection
VTA [18] 2019	31.25	Pytorch Tensorflow	ResNet, VGG	Classification
[19] 2021	59.78	Pytorch	VGG	Classification
[20] 2020	17.9	Tensorflow	MobileNet SqueezeNet	Classification
[21] 2021	16.04	N/A	MobileNet	Classification

demonstrating DPU’s superior performance and scalability. As an acceleration engine, DPU seamlessly coordinates with other system components, making it attractive to design houses and industries. For example, DPU is widely used in automotive, medical, and industrial applications, such as Baidu Apollo [13], Subaru [14], and SplineAI [15]. Note the hardware description language (HDL)-based DPU soft IP is encrypted to protect confidentiality.

Model architecture retrieval is hard without knowledge of the underlying hardware resources and their support for varying layers and network model structures. For finer-grained attacks, such as model parameter extraction using Correlation EM Analysis (CEMA), an accurate leakage model for the target accelerator is essential, depending on the precise timing of arithmetic operations. Currently, the scheduling scheme/data flow of DPU is unknown, and the HDL files are encrypted [6], [16]. Considering the widespread use of AMD-Xilinx DPU and FPGAs, assessing the security vulnerabilities of DPU could have substantial real-world impacts. This work makes several contributions as follows:

- We design a reverse-engineering flow that reveals the encrypted structure and scheduling scheme of a DNN accelerator IP. The flow is demonstrated using AMD-Xilinx DPU, a commercial, state-of-the-art, and encrypted DNN FPGA accelerator soft IP.
- We innovatively employ EM side-channel analysis to assist in reverse engineering, addressing numerous challenges associated with accelerator-supported operations, including input partitioning, channel-level parallelism, pixel-level parallelism, and spatial mapping of input/output channels and hardware.
- Enabled by the insights and knowledge gained from reverse engineering, we construct a cycle-accurate simulator, *sDPU*, that tracks the precise clock-cycle-wise state of all DPU registers, facilitating further attacks.
- We explore new attack surfaces exposed by the reverse-engineering results. We conduct a model architecture recovery attack to retrieve the structure and hyperparameters with high fidelity.
- Leveraging *sDPU*, we develop a parameter-extraction attack using EM side-channel analysis. To the best of our knowledge, this is the first successful attempt to accurately recover the model weights from a real-world, high-throughput FPGA DNN accelerator.
- To showcase the versatility of the reverse-engineering process, we apply it to another FPGA-DNN accelera-

tor, TVM-VTA [18], on a different FPGA board, and successfully recover the data flow and schedule as well.

2. Background and Related Work

2.1. FPGA DNN Accelerator

In advanced FPGA DNN accelerators like AMD-Xilinx DPU and TVM-VTA, the instruction-flow-based design principles have gained prominence due to their versatility in implementing various DNN models using a single bitstream [22]. DNN models often contain computation-intensive layers, such as the convolution layer and fully-connected layers, which are rich in operations that can be executed concurrently for accelerated performance. In these layers, the input and output channels are independent, and at a finer granularity, the convolutions of different output feature map pixels are also independent. Such independence offers three types of *parallelism* that accelerators can exploit to expedite inference: input channel parallelism (*ICP*), output channel parallelism (*OCP*), and pixel-level parallelism (*PP*).

It is important to note that identical values of *ICP*, *OCP*, and *PP* do not guarantee equivalent performance across DNN accelerators. The intricacies of design and implementation significantly influence performance outcomes. For instance, if one accelerator has an initiation interval (II) of one, while another has an II of three, the performance discrepancy between the two could be about $2\times$.

2.2. Deep Learning Processor Unit

We select the AMD-Xilinx DPU as the target to demonstrate our attack, since it represents one of the most sophisticated FPGA-based DNN accelerator soft IPs available on the market, providing exceptional performance per watt. It has been utilized in various real-world applications, including production vehicles by Subaru [23], [24] and the Apollo autonomous driving system by Baidu [25]. DEEPhi, the company that developed the AMD-Xilinx DPU, was acquired by Xilinx in 2019 for an estimated price of \$300 million [26]. The high efficiency of the AMD-Xilinx DPU can be attributed to its near-optimal hardware-software co-design. For example, when inferring ResNet-18 on ZCU102, the AMD-Xilinx DPU outperforms one of the most prominent open-source FPGA accelerators, TVM-VTA, by a factor of ten [27], [28].

DPU comprises multiple modules that serve different functions, as illustrated in Figure 1 (left section). In this work, we focus on reverse-engineering the scheduler’s functionality and the architecture of the DPU’s hybrid-computing array. Throughout the paper, we refer to the DPU itself as the *DPU soft-IP* and the FPGA hardware loaded with the compiled bitstream of the DPU soft-IP as the *DPU accelerator*.

DPU supports seamless deployment of DNN models using popular development frameworks such as TensorFlow and PyTorch. Users can directly compile DNN models trained by these frameworks into *elf* models for the DPU using

the proprietary, closed-source compilation tool Vitis-AI [5]. AMD-Xilinx maintains a model zoo with over one hundred pre-compiled elf models, including popular models like VGG [29], ResNet [30], GoogLeNet [31], YOLO [32], and MobileNet [33].

As a commercial FPGA accelerator IP, the confidentiality of the DPU has been stringently protected at the source code (Verilog) level. Although users can compile the DPU soft IP into a bitstream and deploy it on an AMD-Xilinx FPGA, they do not have access to the internal details. According to AMD-Xilinx [34], the entire end-to-end development process, encompassing HDL design, simulation verification, synthesis, and bitstream generation, adheres to the IEEE-1735-2014 V2 standard [35] and cannot be modified [36]. It should be noted that in this work, we do not attempt to directly crack the soft IP encryption. **Instead, we rely solely on public information and EM side-channel analysis to extract the design and implementation details of the DPU.**

2.3. EM Side-channel Analysis

Side-channel analysis (SCA) exploits information leakage like sound, power, latency, and electromagnetic emissions from the victim system to extract secret data. Simple EM analysis (SEMA) links EM signals to operations, recovering private keys in ciphers like ECC and RSA. Correlation EM analysis (CEMA) gathers numerous EM traces, analyzes statistical leakages, and uses data dependence of EM signals to recover secrets. Attackers inspect the device under test (DUT) operations and structure to build a suitable EM model for a module, such as a register holding an intermediate computation value dependent on the secret. After identifying the corresponding time point on the EM trace when the register updates with the intermediate computation value, the attacker correlates the EM values predicted by the model with the measured EM signals. Hypothesis testing is conducted for all possible secret values, with the correct value producing the highest correlation. Two common EM models are Hamming weight (HW) and Hamming distance (HD). HW counts ones in the binary representation of v_i , while HD measures bit toggling between two consecutive register values: $HD(v_{i-1}, v_i) = HW(v_{i-1} \oplus v_i)$.

2.4. Related Works

Several studies have investigated attacking DNN models on FPGA DNN accelerators with varying objectives. Wei et al. [37] utilized power analysis to recover the input from a binary neural network. Similarly, Moini et al. [12] employed remote power analysis to achieve the same goal. Other works have also attempted to recover the network architecture of neural networks running on accelerators [38], [39], [40]. Some prior work [10], [11] has targeted systolic-array-based FPGA neural network accelerators to extract parameters, such as weights and biases, using power analysis. These studies either employed open-source or handcrafted FPGA accelerators, which assume the attackers own complete

knowledge of the internal hardware structure and target scheduling.

To the best of our knowledge, our work is the first attempt to reverse-engineer a commercial, closed-source, encrypted FPGA DNN accelerator soft IP and its operation scheduling. Gaining knowledge of these technical details, we further explore new attack surfaces, demonstrating the security implications with a DNN model architecture recovery attack and a model parameter extraction attack.

2.5. Threat Model

Our proposed attacking process comprises two phases. The first phase is reverse-engineering the encrypted soft IP, where we deploy DNN models on DPU accelerators to conduct EM SCA. The adversary's capabilities *during the reverse-engineering phase* are summarized as follows, s/he can:

- Compile the encrypted soft IP into a bitstream and load it onto an FPGA board.
- Train any DNN models and generate corresponding binary files.
- Measure the EM leakage of the accelerator during model inference.
- Access the publicly available manual.
- Possess knowledge of and access to the tool used for bitstream generation.

The second is the attack phase, which leverages the reverse-engineering results to recover the architecture and parameters of unknown DNN models running on the DPU accelerators. The adversary's capabilities *during the attack phase* are summarized below:

- Utilize knowledge gained from the reverse-engineering phase.
- Measure the EM leakage of the accelerator during model inference.

3. Overview of Encrypted DNN Accelerator IP Reverse-Engineering

DNN accelerator IPs are often developed following a software-hardware co-design principle, which aims to effectively and efficiently harness the potential for execution parallelism, a challenging task given the vast design space and the limited hardware resources. While constructing a functional accelerator is relatively straightforward, developing an optimized one is very challenging. Since DNN accelerators typically process neural networks on a layer-by-layer basis, i.e., the output generated by each layer of a DNN serves as the input for the subsequent layer, buffering the output to facilitate computations for the next layer.

The accelerator's hardware design determines the level of parallelization and data flow, which includes execution paths of the input feature maps, model parameters, and output feature maps. The software component of a DNN accelerator is responsible for scheduling, which involves

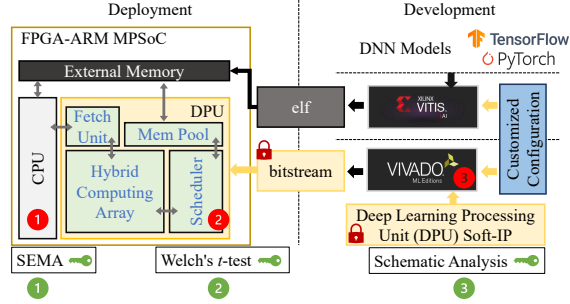


Figure 1. **Right:** The development and compilation process for implementing a DNN model on DPU. **Left:** DPU top-level block diagram from [6].

segmenting the workload of each layer based on the underlying parallelization capability. Moreover, the software must schedule the operations within each segment to align with the hardware components. Based on this understanding, our reverse-engineering flow comprises two parts: (1) recovering the scheduling for various sizes of input feature maps and model parameters, and (2) reverse-engineering the hardware hierarchy, with the former dependent on the latter.

We illustrate our reverse-engineering flow in Figure 1, including the involved software or hardware components and the techniques employed. Specifically:

① $\Rightarrow$ ① We recover intra-layer workload segmentation, i.e., how to partition the workload into segments that fit on the hardware, using simple electromagnetic analysis (SEMA), described in Section 4.2.

② $\Rightarrow$ ② We recover the intra-segment scheduling using Welch's t -test, illustrated in Section 4.3.

③ $\Rightarrow$ ③ We reverse-engineer the hardware architecture and data path using schematic analysis, in Section 5.

Finally, with both the scheduling and data path discovered, we combine these two parts, derive the scheduling algorithm, and evaluate the results in Section 6.

Throughout the paper, we apply our reverse-engineering flow to the AMD-Xilinx DPU. To demonstrate the generality of our approach, we also reverse-engineer another DNN accelerator IP, TVM-VTA, using a different FPGA board, and the results are presented in Appendix C.

4. Reverse Engineer Software Scheduling

What we can acquire from the manual of the DNN accelerator is only the configuration, specifying the input channel parallelism (*ICP*), output channel parallelism (*OCP*), and pixel parallelism (*PP*). These numbers are important features of DNN accelerators pertaining to parallelization. However, they do not offer precise information about how the workload is divided within a layer, such as the segment size, the cycles required for each segment, and handling workload sizes larger than the segments. The objective of intra-layer workload segmentation is to recover such details. We employ the SEMA technique and design a workflow to achieve the goal. After obtaining sufficient segmentation details, we commence reverse-engineering of the intra-segment scheduling, i.e., the

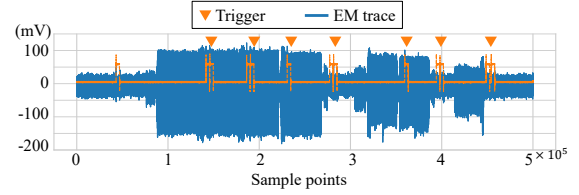


Figure 2. A sample EM trace with write-back signals.

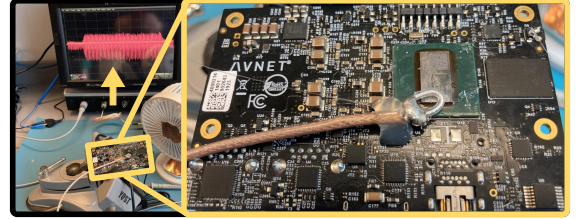


Figure 3. The experimental setup for EM trace collection.

fine-grained timing of arithmetic operations within a segment. We accomplish this by carefully selecting a series of inputs and weights and obtaining the timing information using Welch's t -test on EM traces.

4.1. EM Trace Collection and Segmentation

Methodology:

EM traces are collected when a DNN model executes on an accelerator with certain input. To infer semantic (model operation) information from the EM traces, the first step is to separate and align the EM traces by segments for further analysis. However, precisely segmenting the EM traces poses a challenge. We attempted various signal processing methods, including bandpass filter, dynamic time warping, and wavelet transformation, which are effective trace alignment methods employed in prior side-channel analyses of cryptographic execution. Due to the high noise, lengthy trace, and significant jitters on the Advanced eXtensible Interface (AXI) data transmission, none of these methods yielded ideal trace alignment for EM analysis. We ultimately utilized the control signals of the AXI buses for segmentation, which are external to the encrypted IP. These signals exist because the on-chip buffer cannot accommodate all intermediate data when the workload surpasses one segment. The accelerator needs to signal the off-chip memory to transmit the data, making them ideal for separating EM trace segments.

Demonstration:

For the AMD-Xilinx DPU, we select the write-back signal as the trigger. Figure 2 illustrates an EM trace and its triggers using the write-back signals. Each segment is followed by varying numbers of write-back signals, depending on the output size of the segment. The start of each group of write-back signals provides an ideal timing reference for both trace segmentation and operation alignment. We select the Ultra96-v2 development board [41] as the target experimental platform, which is popular and generic. We install the PYNQ

v2.6 operating system driven by the PetaLinux kernel [42], [43]. We deploy DNN models supported by the DPU on the target platform. The DPU configuration specifies the number of computing resources. We choose the largest configuration B1600 that fits on the FPGA. For reproducibility, we list the detailed configuration in Table 5 in the Appendix.

Figure 3 presents our experimental setup. We utilize an EM probe with a pre-amplifier from Aaronia AG [44] to capture EM signals and convert them into voltages. The EM probe is positioned above the emanating areas of the Device Under Test (DUT) and connected to a Lecroy oscilloscope [45]. The DUT and the oscilloscope are linked to a host workstation for command execution and synchronization. To collect an EM trace, the host first sends a command to the DUT to initiate DNN inference, while simultaneously, the oscilloscope captures EM signals streamed to the workstation. We set the sample rate to 10 GS/s for the experiments.

4.2. Intra-layer Workload Segmentation

Methodology:

For SEMA, we obtain one EM trace for each DNN model inference. We then examine the similarities and differences between these traces to extract information about the DNN model execution. Our analysis primarily targets the high-amplitude segments of an EM trace, specifically focusing on duration, amplitude, and distance to other segments. To maximize EM emission from the computational part, we set every pixel of the input to random values. We then scan the DUT using the EM probe and locate a probing point such that the high-amplitude portions of the EM traces correspond to computation, while the low-amplitude sections in between represent data transmission. This distinction is feasible because data transmission typically consumes less power. Clear data transmission between layers allows for easy separation of each layer, with segment separation provided by the trigger signal.

Next, we determine how the accelerator segments each layer based on input height H , input width W , the number of input channels IC , and the number of output channels OC .

Step 1: Initially, we investigate the influence of H and W on segmentation. We compile the DNN such that the target layer has IC and OC less than or equal to ICP and OCP , respectively. We first identify a pair of small W and H values, ensuring there is only one segment for the target layer. Then, we fix H and incrementally increase W , observing its relationship with the number of segments N . We repeat this process for H .

Step 2: Subsequently, we explore the impact of IC and OC on N . Following a similar procedure as the previous step, we fix H and W at small values and observe how IC and OC affect N independently.

Step 3: Since kernel width K_w and kernel height K_h are typically much smaller than W and H , they are not segmented. These parameters influence the duration (number of clock cycles) of a segment. After completing the previous two steps, we have determined the workload capacity of the

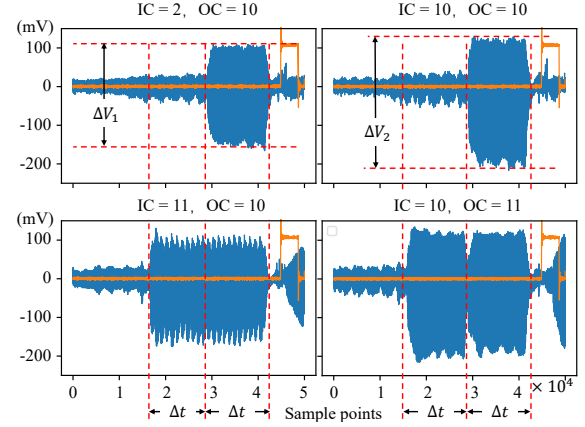


Figure 4. EM traces of various input and output channels; each trace (blue) is averaged over the same input 100 times to reduce noise. Traces across each measurement are aligned using the trigger signal (orange).

segments. We now investigate the number of clock cycles required for this workload. As in the previous steps, we adjust one parameter at a time and measure the changes in clock cycles.

Demonstration:

Next, we demonstrate intra-layer workload segmentation through SEMA. In the case of the AMD-Xilinx DPU, $ICP = 10$, $OCP = 10$, and $PP = 8$. Figure 2 displays an EM trace corresponding to the execution of a three-layer neural network. We can easily identify three high-amplitude parts, each corresponding to a distinct layer. We can now proceed with the segment analysis as described in the methodology above.

Step 1: We observe that only the input height affects the number of segmentations. Our observations are formulated below:

$$N = \lceil \frac{H}{H_b} \rceil \quad (1)$$

This means that the input is segmented horizontally, where $H_b = 8$ is the block size.

Step 2: As depicted in Figure 4 (the top plots vs. the bottom plots), when either the number of input or output channels exceeds 10, the duration of each segment doubles. We formulate these experimental observations as follows:

$$N = \lceil \frac{IC}{ICP} \rceil \lceil \frac{OC}{OCP} \rceil \quad (2)$$

This result suggests that the hardware structure can only handle the workload at once when both IC and OC do not surpass ICP and OCP . Otherwise, the hardware is reused in an unrolling fashion.

Step 3: As demonstrated by the previous steps, the height of the input does not affect the length of the segment. We formulate the relationship between the length of the segment and the parameters below, where Δt is the duration of the segment, s is the stride size, f is the operating frequency of the accelerator, and C is the number of clock cycles.

$$\frac{\Delta t}{f} = C \approx \frac{W - K_w + 1}{s} K_w \cdot K_h \cdot 2 \quad (3)$$

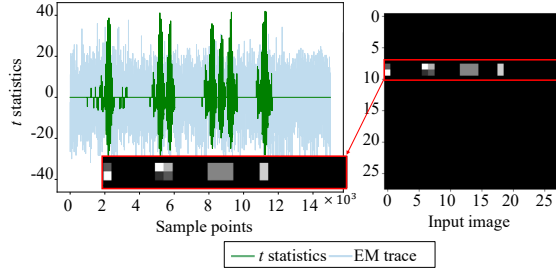


Figure 5. T-test to determine the time and duration of each input column.

We observe another feature of the AMD-Xilinx DPU: the amplitude of the high-amplitude sections also leaks information. For example, in the top two plots in Figure 4, we can observe two amplitude levels corresponding to different numbers of input channels. When the IC is less than or equal to 5, the amplitude remains at one level ΔV_1 ; while the amplitude is considerably higher when the IC is between 6 and 10, $\Delta V_2 > \Delta V_1$. This observation indicates that the hardware may have two blocks, each supporting up to five input channels.

4.3. Intra-segment Scheduling

Intuition:

After recovering the intra-layer segmentation, we know exactly the workload of each segment. In this section, we recover the fine-grained scheduling within each segment using Welch's t -test. Welch's t -test is a statistical test to see if two groups of data have equal means. It is widely used in conventional side-channel analysis to evaluate leakages of cryptographic ciphers [46].

The key idea of employing Welch's t -test on EM traces is to identify the time point at which an arithmetic operation of interest happens. We prepare two datasets: the first dataset consists of inputs of all zeros, and the second dataset has non-zero values strategically placed on the input, as can be seen in the right section of Figure 5. We collect EM traces of both datasets. For each sample point of the EM traces, we perform Welch's t -test over the two datasets to determine if they have the same mean. If the t -test shows that the mean of the EM amplitude is different between the two datasets, it indicates that the difference is due to *different input features*. More specifically, it means that the DUT is processing *the specific input feature* at those time points. For example, looking at the green t -statistics on the left section of Figure 5, we can see that each column of input corresponds to a spike of t -statistics (the value is larger than 4.5), meaning that DUT processes the input column by column from left to right.

Methodology:

We have shown in the intuition that we can determine the time point and duration of specific arithmetic operations using Welch's t -test. Now, we identify how the accelerator schedules the rows/columns of input, rows/columns of the kernel, and input and output channels within a segment. For

steps 1 and 2, we set IC and OC both to be one. For step 3, we set $IC = ICP$ and $OC = OCP$.

Step 1: First, we investigate the processing order of the input, i.e., whether the accelerator handles the input segments in row-major or column-major order. We set the kernel's width and height to both be one. Then, we perform a t -test with input with a non-zero row and a non-zero column, respectively. If the DUT is row-major, we will see one t -statistic spike with the non-zero-row input. Meanwhile, there will be spikes across the segment with the non-zero column. If the DUT is column-major, we will observe the opposite.

Step 2: Next, we determine the order of the kernel. We set the input accordingly: if the DUT is column-major, we set the input to have a non-zero column; otherwise a non-zero row. We then set the kernel to have a non-zero row and column, respectively. We will observe sparse and dense t -statistic spikes. The order that produces dense spikes is the order of the kernel.

Step 3: Finally, we find the scheduling of the input and output channels. Similar to step 2, we set the input to have a non-zero row or column according to its order. Then, for each of the input channels and output channels, we set one and only one filter to be non-zero. Then, we can determine the starting time of each channel by comparing the starting time of the corresponding t -statistic spikes.

Demonstration:

We demonstrate the results of these steps on AMD-Xilinx DPU.

Step 1: As can be seen in Figure 5, DPU's input order is column-major.

Step 2: DPU's kernel order is row-major.

Step 3: DPU's input channels 0...4 each start one cycle later than the previous one. Input channels 5...9 also start one cycle later than the previous one. Input channel 5 starts at the same time as channel 0. Output channels 0...10 each start one cycle later than the previous one. Output channel 0 starts at the same time as input channel 0.

5. Reverse Engineer Hardware Hierarchy

Methodology:

We employ two approaches, schematic analysis, and template matching, to reconstruct the hardware hierarchy. Although the IP source code is encrypted, the Vivado tool explicitly permits users to examine the schematic and hierarchy of encrypted IPs during synthesis. This is common, as Electronic Design Automation (EDA) vendors, including AMD-Xilinx and Cadence, generally disclose encrypted IP core schematics and netlists during synthesis for various purposes, such as verification and performance/power simulations. Intel, however, opts not to exhibit the internal schematic of encrypted IPs. Nonetheless, third-party tools exist to enable schematic view [47]. By default, component names remain concealed [48]. We circumvent this confidentiality by leveraging access to the hierarchy. In our experiment, when compiling the same Verilog module multiple times, they maintain the same hidden name, a random string. This permits us to ascertain how often a module is employed and

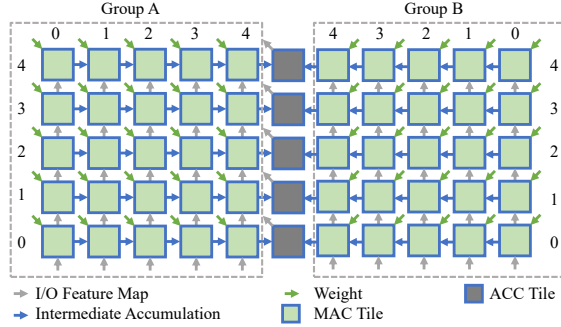


Figure 6. The core hardware structure of the encrypted DPU IP.

the manner in which these modules are interconnected. We concentrate on blocks with a greater number of DSP slices, as we know that multiplication and accumulation transpire in these DSP slices.

EDA tools extensively optimize circuits during synthesis, rendering it challenging to identify even fundamental modules such as an adder. We address this constraint by creating a library of schematic templates for arithmetic operations (including addition, accumulation, and multiplication) through RTL synthesis and applying them for pattern recognition in the schematic of the DUT.

Step 1: Since the hardware is designed for parallelization, numerous basic components possess identical functionality. In this step, we identify components comprising DSP slices and compose the block diagram.

Step 2: We ascertain the functionality of basic components using template matching.

Step 3: Upon recovering the workload size for each segment, we can deduce the data path for input, kernel, and result based on data flow direction and bandwidth.

Demonstration:

We demonstrate these steps on the AMD-Xilinx DPU.

Step 1: Using the hierarchy information alone, we identify two major blocks with 200 and 80 DSP slices, respectively, as shown in Figure 6. The 200-DSP block has two identical groups, A and B. Each group has a five-by-five tile array with data flowing through them, and each tile contains four DSP slices. The 80-DSP block is composed of five accumulation tiles.

Step 2: Using template matching, we discover that there are two types of tiles. One performs multiplication and accumulation, which we call the MAC tile. The other performs accumulation only, which we call the ACC tile.

Step 3: For a MAC tile, two essential inputs are required: input and weight. We determine the bus width for each input, which amounts to 400-bit and 640-bit, respectively. Given that the input feature maps exhibit a larger size than the kernel in each segment, we infer that the 640-bit input from row 0 corresponds to the input feature map. The 400-bit input is also reused by the other rows, which further confirms the finding that it is the input feature map. The 400-bit input for every MAC tile represents the weight. The MAC tile's

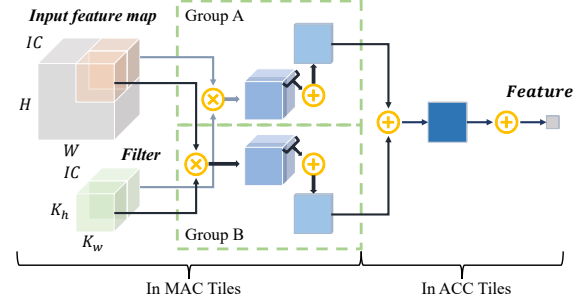


Figure 7. Illustration of how the DPU processes an input feature map when there are 10 input channels and two output channels.

additional input is the accumulation input, while its output is the accumulation output.

In summary, following these three steps, we establish that Group A and Group B are identical. Each group processes inputs entering row 0, carries out multiplications and accumulations within MAC tiles, and releases outputs through rows in columns 4 and 5. The bottom row receives input from the input buffer, storing it for one cycle and operating on it before transferring it to the upper row. This results in a one-cycle delay in inputs for two rows. Each MAC tile multiplies the input with weights and accumulates the product with the resulting output from the adjacent feeding column. Outputs from both groups are accumulated by another block of 80-DSP slices in five accumulation (ACC) tiles. In each group, column 0 solely performs multiplication, and the accumulation is disregarded (considering the input to add as zeros). Each tile also features internal buses for routing the operands (inputs and weights) for computation.

Combining with the scheduling information in Section 4.3 we can easily see that each column takes one input channel and each row handles two output channels. We illustrate how the DPU processes a workload segment. With this highly regular computation across the columns (over different input channels) and the rows (with different filters) of the DPU accelerator, one row of computation across the two groups is shown in Figure 7.

A notable finding of the DPU is, surprisingly, the Vivado synthesis flow uncovers critical metadata of DSP slices. Vivado exhibits the OPMODE (denoting the arithmetic function) and INMODE (input selection) of the DSP slices, which provide sufficient information for us to reconstruct the entire configuration of the DSP slices. We observe that the configuration of other non-DSP hardware units, such as fast carry logic (CARRY8), remains concealed. This enables our methodology to function without necessitating template matching when recovering the functionality of the DSP slices.

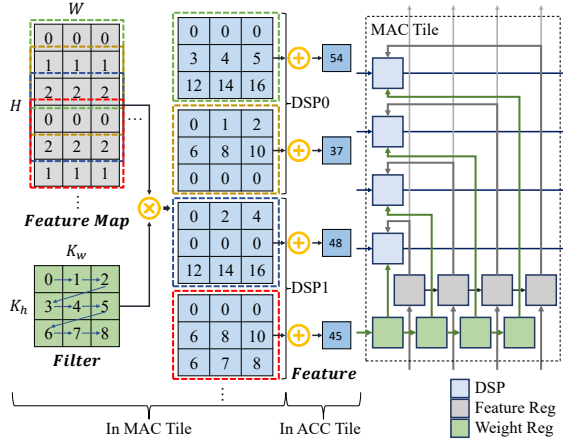


Figure 8. Illustration of how a MAC tile works

6. Reverse Engineering Results and Evaluation

6.1. Putting All Things Together: Mapping the Workload to the Hardware

In this section, we present the reverse-engineered scheduling algorithm, which is used to construct the DPU simulator sDPU, providing a foundation for the two attacks discussed in Section 7. The DPU accelerator, as illustrated in Figure 7, carries out highly regular computations across columns (different input channels) and rows (distinct filters). Consequently, the functionality of each Multiply-Accumulate (MAC) tile remains uniform, and the DPU's scheduling essentially corresponds to the scheduling of a MAC tile. Figure 8 demonstrates the operation of a MAC tile.

A MAC tile processes a single input channel, with both the feature map and the filter being two-dimensional, represented in gray and green, respectively. For the purpose of illustration, the filter size is set at 3×3 . The example includes a partial feature map and two Digital Signal Processors (DSPs) out of the four in a MAC tile, indexed from 0 to 3. The schematic of a MAC tile is shown on the right side of Figure 8. Each DSP slice conducts element (scalar) multiplications and accumulations to generate two output features. In each cycle, a DSP slice loads two input feature values (on the same column and from two adjacent rows) and concatenates them into one register. Additionally, it loads a corresponding weight value from the filter and multiplies it with the register containing the two input features. This product is added to the accumulation result from the MAC tile to its left by its corresponding DSP at the same index. The updated accumulation result is then stored in a register.

During the next clock cycle, the DSP slice in the subsequent right MAC tile, with the same index, retrieves the accumulation result and adds it to its local product of another channel feature and weight. The accumulation across the MAC tiles in a row achieves the sum-reduction across the input channels depicted in Figure 7. Since the DSP must accommodate two filters of two output channels, it loads

the same input features for two consecutive clock cycles. In the first cycle, the DSP loads the weight from one filter, and in the other cycle, the weight is one that is at the same coordinate as the other filter. Both the input feature loading and the weight-loading sequences follow a row-major pattern, as shown in the filter of Figure 8.

To calculate the number of cycles required for a MAC tile to process three columns of input features, we must process all the weights in the two filters, resulting in $K_w \cdot K_h \cdot 2$ cycles. It is important to note that the weight registers are chained, meaning DSP 3 does not become active until 3 cycles after DSP 0 is activated, with the four DSPs in one MAC tile operating in a pipelined manner. Consequently, the number of required cycles must also account for pipeline filling time, or $K_w \cdot K_h \cdot 2 + 3$. The feature map's height does not contribute to the cycles, as the DPU divides the workload into batches, processing batches of input height in parallel. This can be easily extended to the number of cycles needed for all the tiles in a row to process three columns of input features by simply adding 4 to the number of cycles, considering the pipelining fashion across the columns. The formula for any input width W is as follows:

$$C = \frac{W - K_w + 1}{s} K_w \cdot K_h \cdot 2 + 3 + 4 \quad (4)$$

Note this equation matches the reverse engineering result shown in Equation 3, except for the two constants considering the pipeline, where 3 denotes the number of pipeline stages within a tile (with four DSPs), and 4 signifies the number of pipeline stages of a row (with five input channels).

We now assemble the schedule for a row of MAC tiles. In the algorithm, we use underline to represent a variable. Algorithm 1 takes two inputs for a layer: Feature and Filter, and schedules two features and a weight into the DSPs of the target row in each cycle. The input Feature has three dimensions: input channel, width, and height. The input Filter comprises four dimensions: output channel, input channel, kernel width, and kernel height. The output is a three-dimensional array Stream, with dimensions of cycle, MAC tile (column) index, and DSP index within, respectively. Each entry in the array is a data structure consisting of two input feature values and one kernel weight value. The Stream is initialized with all zeros; therefore, if the algorithm does not specify data for some DSPs at certain cycles, the DSPs will process all-zero input and weight values at those cycles.

The three loops from lines 14 to 16 traverse each cycle and assign weights and input features to each DSP. Each DSP does not initiate computation until the previous DSP has the result ready for accumulation, i.e., one cycle later than the previous one in its left tile. As seen in Figure 8, in each MAC tile, the weight is buffered for one cycle before entering the next tile, resulting in each MAC tile commencing a cycle later than its predecessor. Combining these observations, the cycle during which a DSP begins computation is given by $\text{col} + \underline{d}$, where col is the column index of the tile within the block, and $\underline{d}$ is the index of the DSP in the MAC tile.

Subsequently, we determine which weight and inputs are fed to the DSP. Lines 21-23 outline the row-major pattern

Algorithm 1 Scheduling the weight and input for row 0

Require: Feature, Filter

- 1: $\triangleright$ (Feature, 3D array)(IC, W, H): input feature map $\triangleleft$
- 2: $\triangleright$ (Filter, 4D array)(OC, IC, K_w, K_h): filters, $OC \leftarrow 2$ $\triangleleft$

Ensure: Stream

- 3: $\triangleright$ (Stream 3D array):(cycles, nCol, nDSP) the data stream that contains weight and input feature for each DSP at each cycle
- 4: $nCol \leftarrow 5$ $\triangleright$ there are five columns of MAC tiles in each group
- 5: $IC \leftarrow nCOL$ $\triangleright$ Each column can support one input channel
- 6: $nDSP \leftarrow 4$ $\triangleright$ there are four DSP slices in each MAC tiles
- 7: $InPerDSP \leftarrow 2$ $\triangleright$ DSPs pack two input values in the input register
- 8: $\triangleright$ Number of cycles to complete one row of a kernel $\triangleleft$
- 9: $CyclePerKernelRow \leftarrow OC \cdot K_w$
- 10: $\triangleright$ Number of cycles to complete all rows of a kernel $\triangleleft$
- 11: $CycleAllKernelRow \leftarrow CyclePerKernelRow \cdot K_h$
- 12: $\triangleright$ number of cycles to process this feature map $\triangleleft$
- 13: $cycles = OC \cdot (W - K_w + 1) \cdot K_w \cdot K_h + (nCol - 1) + (nDSP - 1)$
- 14: **for** $c=0$ to $cycles$ **do**
- 15: **for** $col=0$ to $nCol$ **do**
- 16: **for** $d=0$ to $nDSP$ **do**
- 17: $sCycle \leftarrow col + d$
- 18: $\triangleright$ Each DSP starts to compute at a different clock cycle so that the DSP on its left has the result ready for accumulation $\triangleleft$
- 19: **if** $sCycle \geq c$ **then**
- 20: $rCycle \leftarrow (c - sCycle) \triangleright$ the number of cycles that the DSP is active
- 21: $k_w \leftarrow (rCycle + CyclePerKernelRow) \bmod K_w$
- 22: $k_h \leftarrow (rCycle + OC) \bmod K_h$
- 23: $w \leftarrow k_w + \lfloor rCycle / CycleAllKernelRow \rfloor$
- 24: **if** $w \geq W$ **then**
- 25: $\perp$ continue
- 26: $h \leftarrow d \times InPerDSP + k_h$
- 27: $oc \leftarrow rCycle \bmod OC$
- 28: $Stream[c][col][d].in[0] \leftarrow Feature[col, w, h]$
- 29: $Stream[c][col][d].in[1] \leftarrow Feature[col, w, h+1]$
- 30: $Stream[c][col][d].w \leftarrow Filter[oc, col, k_w, k_h]$

that the DSP slice follows to load the weights. The input feature row index h is determined solely by the weight index and DSP index. The input feature column index w is further determined by the number of previously processed columns. Two output channels must share this row, which is achieved by computing the first output channel in even-numbered cycles and the second one in odd-numbered cycles. The weights from the two output channels multiply with the same input features.

6.2. Evaluation Benchmark

In this subsection, we introduce the evaluation benchmark that will be used in the subsequent experiments. The benchmark examines compute-intensive operators such as Conv2D and Dense workloads from the Xilinx Model Zoo [17]. This repository contains an extensive collection of DNN applications executable on DPU, including 130 models from over 40 architectures, covering a wide range of tasks. The models can be categorized based on their tasks as follows: (1) Image classification models, such as VGG, ResNet, Inception, and MobileNet; (2) Image object detection models, including YOLO, SOLO, SSD, and EfficientNet; (3) Image object instance segmentation models like Mask R-CNN and U-Net; (4) Image super-resolution models, for example, RCAN; (5) Natural Language Processing (NLP) models, such as BERT;

TABLE 2. CONV2D WORKLOAD BENCHMARK EVALUATION RESULTS.

Conv2D Workload					EM trace Measured		sDPU
IC	OC	$K_{h/w}$	$IF_{h/w}$	$OF_{h/w}$	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
16	16	3	112	112	348425	108883	111496
32	64	3	147	73	4357412	1361692	1394372
64	16	1	112	112	143937	44981	46060
128	64	3	112	56	7926668	2477084	2536534
512	384	3	14	14	2877225	899133	920712

TABLE 3. DENSE WORKLOAD BENCHMARK EVALUATION RESULTS.

Dense Workload					EM trace Measured		sDPU
IC	OC	$K_{h/w}$	$IF_{h/w}$	$OF_{h/w}$	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
512	1000	1	1	1	211250	66016	67600
768	768	1	1	1	240865	75271	77077
1024	1000	1	1	1	418437	130762	133900
1536	1000	1	1	1	625625	195508	200200
4096	1024	1	1	1	1715593	536123	548990

and (6) Vision Transformer models. It is worth noting that some NLP models do not rely on Conv2D; however, their Linear layers can be represented in the Dense format, as demonstrated in our paper. After examining all the models in the Model Zoo, we extracted over 300 different configurations of Conv2D and 23 configurations of Dense.

6.3. Experimental Evaluation: A Simulator

We evaluate the effectiveness and accuracy of our reverse engineering of the DPU. Based on the insights gained, we construct a DPU simulator, *sDPU*, following Algorithm 1, which can simulate the status of all DPU internal registers at the resolution of each clock cycle.

We validate *sDPU* by comparing its simulations with an actual DPU processor. Specifically, we use a Conv2D workload with dimensions [10, 10, 3, 3, 8, 28, 6, 26] as a case study, where the configuration tuple corresponds to the output-channel size (OC), input-channel size (IC), kernel height and width (K_h, K_w), and the height and width of the input and output feature maps (IF_h, IF_w, OF_h, OF_w), respectively.

We first compare the outputs between *sDPU* and a real DPU, which match and verify the functional correctness of *sDPU*. We also depict the internal PE status simulated by *sDPU* for this Conv2D workload shown in Figure 9, which takes a total of 478 clock cycles. In the simulation results of *sDPU*, the input feature and weight are represented as $[IC, IF_h, IF_w]$ and $[OC, IC, K_h, K_w]$, i.e., 3-digit and 4-digit numbers, respectively.

To verify that the simulation resolution is as accurate as clockwise, we apply the same Conv2D workload on a real DPU and collect EM traces, from which we can deduce the exact execution time. As illustrated in the top-right plot of Figure 9, the EM trace indicate the DPU execution time of the Conv2D workload is 1493.75 ns. As the DPU is executing at 320 MHz, the total number of clock cycles (cc) consumed is 478, which is in accordance with the simulation results of the *sDPU*.

To thoroughly assess the accuracy of *sDPU*, we evaluate all Conv2D and Dense operators in our benchmark, as shown in Section 6.2. We showcase a selection of 5 Conv2D and

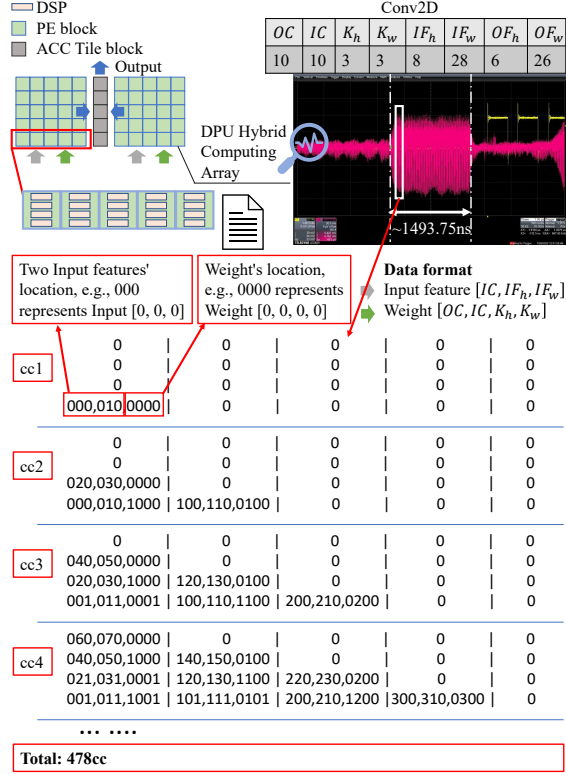


Figure 9. *sDPU* visible simulation report vs. the DPU EM leakage.

Dense configurations, comparing the executions of DPU and *sDPU* in Table 2 and Table 3, respectively. Additional results can be found in Appendix D. The 6th column of Table 2 and Table 3 reports the DPU latency measured from EM traces, while the 8th column presents the *sDPU* performance, i.e., the total clock cycles of a specific workload simulated by the simulator.

Our findings indicate that *sDPU* can deliver clock cycles remarkably close to the DPU, with an error rate $< 1\%$. We argue that due to the AXI protocol, which cannot guarantee constant transmission time for the same amount of data, the error lies in deducing the number of cycles from EM measurements. We are confident that our *sDPU* accurately estimates the clock cycles for each workload. The accuracy of *sDPU* is further corroborated by the parameter-extraction attack using CEMA in Section 7.2, which would not succeed if *sDPU* were off by even a single clock cycle.

7. Reverse-Engineering-Enabled Attacks

7.1. Model Architecture Recovery

In this section, we demonstrate how to recover the DNN model architecture running on a DPU accelerator, utilizing reverse-engineering knowledge aforementioned. To the best of our knowledge, this is the first attempt to recover the DNN architectures on a closed-source DNN accelerator.

We propose a generic 5-step model architecture recovery flow, shown in Figure 10, with each step targeting different

secrets. For better understanding, we use a LeNet-5 model with 3 input channels (IC) as an illustrative example below, and the EM trace collected for it is segmented in Figure 10 (a).

Step 1. Layer number/type: The most observable DNN layers are those computation-intensive layers such as Conv2D and Dense, as they are separated by long stall time (T_l), as shown in Figure 10(a). We can further differentiate these two layer types. Because the DPU is optimized for convolution layers, Dense layers are treated as convolutional layers with 1x1 kernel, and therefore their EM leakage shows more load/store stall time and much lower amplitude level, ΔV_c , as shown in Figure 10(c), compared with other Conv2D layer processing, as shown in Figure 10(b)(d). Following these rules, we identify the illustrated trace has 2 Conv2D layers followed by 1 Dense layer.

Step 2. Kernel size: For convolutional layers, we have derived Eq. 4 to show how the kernel size affects the batch computation time. The batch computation time is annotated as ② in Figure 10. Considering the DSP execution frequency, the EM sampling frequency, and the number of time points for batch computation, we can calculate the cycle counts. For example, for the first Conv2D layer, $C = \frac{\Delta t_b}{32} \approx 1394$.

We can know W from the 1st Conv2D layer, and the stride size s can also be derived from the trace segments, e.g., the example in Figure 10 (b) and (e) have the same input data size 32×32 but different stride sizes: $s = 1$ and $s = 2$. As a result, they are distinguishable from the stall time between computations, i.e., the longer stall time stands for $s = 2$. We plug W and s into Eq. 4 and get the kernel size as 5×5 starting from the 1st Conv2D layer.

Step 3. Input/Output feature size: In this step, we utilize all revealed information from **Step 2** to directly derive the output feature size, which determines the following layer's input feature size. *For general DNN models with more than two convolution layers, we repeat steps 2 and 3 for each layer.* In our example, we can derive the output feature size of the 1st Conv2D layer as shown in Figure 10 (b) is 28×28 .

Step 4. Channel size: We can approximate the range of the input channel (IC) and output channel (OC) sizes, applying our reverse engineering knowledge. As shown in Figure 10 (b), between two triggers the trace segment is one piece, indicating the OC is not exceeding 10 (see Section 4.2). While in Figure 10 (d), the trace segment between two triggers consists of two equal-length blocks with some data transmission in between, implying that the OC ranges from 11 to 20. Note that it is challenging to directly extract the exact IC and OC for DPU considering the IP hardware structure and the parallelism supported, as shown in Figure 6. This coarse-grained method can estimate the IC and OC with an error within 10. To address this issue, we can use the pre-profiled amplitude results (ΔV), as described in Figure 4 and Section 4.2, to reduce the estimation error of IC and OC to be within 5. For example, as shown in Figure 10 (b) and (d), the ΔV_d is larger than 190mV and below 240mV (ΔV_b), indicating the IC of the 2nd Conv2D layer (also OC of the 1st Conv2D layer) is within 5 and 9.

Step 5. Other non-observable layers and Dense layer:

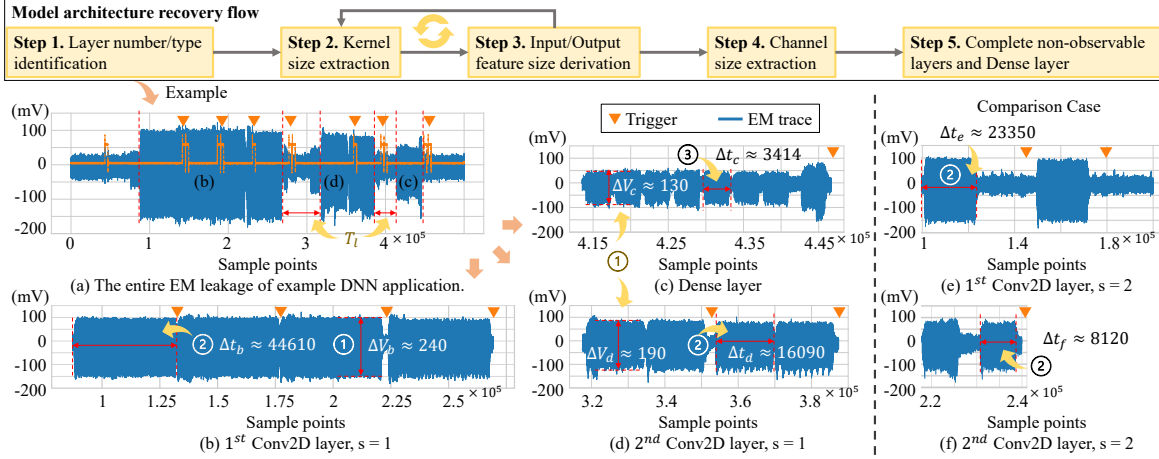


Figure 10. Architecture recovery of a LeNet-5 model. (a) The entire EM leakage. (b) Segment for the 1st Conv2D layer, $s = 1$. (c) Dense layer leakage. (d) 2nd Conv2D layer leakage. (e) Comparison with the 1st Conv2D layer, $s = 2$ leakage. (f) Comparison with the 2nd Conv2D layer, $s = 2$ leakage.

Since DPU highly optimizes the model implementation, the pooling layer becomes non-observable from EM traces. We propose to jointly use the trigger number, Conv2D's stride, and the Dense layer's IC to determine whether there is a pooling layer. For example, we measure the Dense layer $\frac{\Delta t_c}{32} \times 8$ in Figure 10 (c) and find this layer spends ≈ 400 clock cycles (IC of Dense layer is also 400). Considering the output feature size of its previous Conv2D layer is 10×10 , and its channel size is in the range of 11 to 20, we can deduce that there exists a pooling layer reducing the feature map size to 5×5 . Moreover, we can derive that the OC of the previous Conv2D is 16.

Benchmark Evaluation: To evaluate the effectiveness of our approach, we enumerated all benchmarks, as detailed in Section 6.2. We measured the EM trace of each workload and predicted the model architecture, specifically the configuration of each layer, achieving an overall prediction accuracy of 92.88%.

7.2. Model Parameter Extraction

Model parameters, such as weights and biases, are of significant importance and considered the valuable Intellectual Property of models, which are often trained by domain and machine learning experts on proprietary or sensitive datasets, supported by considerable computational resources. Additionally, there are security concerns related to model parameter extraction, as it can enable a wide range of white-box attacks, including adversarial example attacks [49] and membership-inference attacks [50].

We present a novel CEMA side-channel attack against model parameters of the first two convolution layers of a DNN. It is facilitated by insights obtained from the reverse-engineering. To the best of our knowledge, this study is the first successful attempt to recover weights of convolutional neural networks operating on a commercial closed-source FPGA accelerator. Our targeted AMD-Xilinx DPU accelerator is more advanced and sophisticated compared

to those in previous works, as outlined in Table 4. For instance, the DPU operates with the highest frequency and possesses a complex structure, designed to support parallel computations in a highly efficient pipelined manner. These attributes pose challenges in recovering model parameters. Furthermore, the advanced fabrication technology node of the DPU introduces additional challenges for EM-based reverse engineering, as the smaller the technology node the weaker EM emission [51].

Challenge 1: Closed-source, encrypted DUT. Conducting EM analysis is based on an accurate EM model, which can only be established with precise knowledge of the DUT scheduling. In particular, the exact value updating pattern of target registers receiving weight-dependent computation results is required. Without the results from reverse engineering, it is not viable to obtain a meaningful EM model.

Challenge 2: Inadequacy of the Hamming Weight (HW) EM model for DNN secret recovery. Traditional Side-Channel Attacks (SCAs) targeting cryptographic applications, such as AES, have successfully utilized HW EM models for *non-linear* operations, including substitution boxes (SBox). However, in DNN inference, typical operations include multiplications over secret weights and addition with biases, and the HW of their computation results does not exhibit high distinguishability among candidate weight values. For instance, when targeting the DSP PE that performs multiplication, an EM model of $HW(i \times w)$ is over the unknown weight w and the controllable input i . For any input value, the EM model yields identical values when w is a power of two, such as 1, 2, 4, 8, 16, 32, 64, 128 (assuming no computation overflow). Similarly, weight values like 5, 10, 20, 40, 80, 160 also produce the same Hamming weight for multiplication with a given input. For 8-bit multiplications, 191 out of 256 weight values have this HW aliasing issue, owing to the low diffusion of multiplication, a linear transformation.

Challenge 3: High concurrency. Based on our reverse-engineering results, at least 100 DPS slices are activated for models operating on the DPU, irrespective of the size.

TABLE 4. COMPARISON OF EM/POWER WEIGHT EXTRACTION ATTACKS TARGETING DNNs

	Device	Accelerator	Operation Frequency	Technology Node
Batina et al. [52]	Atmel ATmega328P (Microcontroller)	N/A (Serial Program)	20MHz	Not Found
	Atmel SAM3X8E (Microcontroller)	N/A (Serial Program)	84MHz	Not Found
Li et al. [11]	Xilinx Spartan 6 (FPGA)	3×3 Systolic Array	25MHz	90nm
Yoshida et al. [53]	Xilinx Spartan 3A (FPGA)	3×3 Systolic Array	25MHz	90nm
He et al. [54]	ELMO(Simulator)	N/A (Serial Program)	N/A	N/A
Yli-Mäyry et al. [55]	Xilinx Zynq XC7Z020 (FPGA)	GUINNESS (Open source Academia Prototype)	100MHz	28nm
This work	Xilinx Zynq ZU3EG (FPGA)	DPU (Encrypted Soft IP)	320MHz	16nm

Consequently, the side-channel noise is significantly greater compared to serialized operations, such as instructions running on a microcontroller [52], [54] or a simple 3×3 array operating on an FPGA [11], [53].

7.2.1. Correlation EM Analysis (CEMA) Attack. We aim to recover DNN weights as secrets by applying CEMA. For a successful CEMA, it is crucial to identify a suitable EM leakage source (specifically a register) and locate the corresponding time point on the side-channel EM trace. We identify the registers in accumulation tiles as the primary exploitable EM leakage source. We leverage our developed DPU simulator *sDPU* (Section 6.3) to analyze each register involving the secret weights. As the multiplications and accumulations occur in the MAC tiles, it is reasonable to begin with the registers there. As discussed in **Challenge 2**, product registers are not exploitable leakage sources.

The next source to consider is the accumulation registers in the DSP slices. We experimented with EM models of these registers when they are updated with computation results, i.e., $HD(i \times w_0, i \times w_1)$, but our attempts were unsuccessful. With the double rate configuration, one input does not change for two cycles and is supplied to two multiplications with two weights. We suspect such an EM model lacks sufficient diffusion to recover w_0 and w_1 simultaneously with the same input i .

Consequently, the only potential valid leakage source is the accumulation registers in the ACC tiles. In Figure 8, we observe that the ACC tiles accumulate the computation result of the MAC tiles. By setting the model input as random, the accumulation values are diffused enough for correlation EM analysis.

We can precisely locate the target arithmetic operation on the EM trace using our cycle-accurate *sDPU*. This is significantly helpful because searching for the leakage point is expensive, particularly for larger numbers of weights in DNN models. Knowing the time point to target greatly reduces the computational cost of the attack, as we only need to target a single time point for one arithmetic operation.

7.2.2. Attack Implementation. In this section, we demonstrate the parameter extraction attack using a one-input-channel, one-output-channel convolution layer with a 3×3 kernel. With the attack principle validated, it can be extended for more complex layers.

We first show how to recover the first three weights of the first row of the kernel $[w_0, w_1, w_2]$. As depicted in Figure 11, we set the three inputs $[i_0, i_1, i_2]$ in the top left corner of the input image to be random values and all others to zero. Consequently, there are only three non-zero arithmetic operations that occur in the ACC tiles and update the register from the initial zero value to three values consecutively: s_1, s_2, s_3 , where $s_1 = i_0 w_0$, $s_2 = s_1 + i_1 w_1$, and $s_3 = s_2 + i_2 w_2$. We choose the Hamming distance (HD) between s_1 and s_2 , $HD(i_0 w_0, i_0 w_0 + i_1 w_1)$, as the target EM model, because the HD between zero and s_1 is actually the Hamming weight of s_1 , which is not an exploitable EM model. With all the weights unknown, the EM model involves two secret weights w_0 and w_1 . Once EM traces are collected and the time point where the update of the register from s_1 to s_2 occurs is identified, we apply CEMA to the EM traces, with 256×256 weight hypotheses for the pair.

In Figure 11, the two plots on the left show the Pearson correlation coefficients of all the weight guesses under 100K EM traces, where the red curve represents the correct weight value pair, while the rest in gray are for incorrect guesses. The bottom plot displays the rank of the correct guess versus the number of EM traces. With 40K EM traces, the rank of the correct weight guess drops to zero, indicating a successful weight recovery. Now with w_0 and w_1 known, we plug them into the next EM model to recover w_2 , $HD(s_2, s_3) = HD(s_2, s_2 + i_2 w_2)$, where there is only one unknown weight and the enumeration complexity is only 256. Note that this register update happens two cycles after the previous update, i.e., the leaky time point is two cycles apart. Similarly, our CEMA on several time points successfully recovers w_2 , and the rank of the correct weight value quickly converges to zero.

After successfully recovering the first row of the weights, we next demonstrate how to recover the rightmost column of the weights. Similar to attacking the row, we set the top right corner input $[i_3, i_4, i_5]$ to random values and all other inputs to zeros. The weights in the kernel that multiply with these three inputs are $[w_2, w_3, w_4]$. After recovering w_2 , we can recover w_3 using $HD(i_3 w_2, i_3 w_2 + i_4 w_3)$ with 256 guesses as well. Then we can recover w_4 using the same method. We can derive the sequence of kernel weight recovery now: starting from the first row, then the last column, the last row, and the first column, i.e., the outer ring in a spiral-like traversing pattern. We control the input setting (a few non-zero random) correspondingly with the positions of the

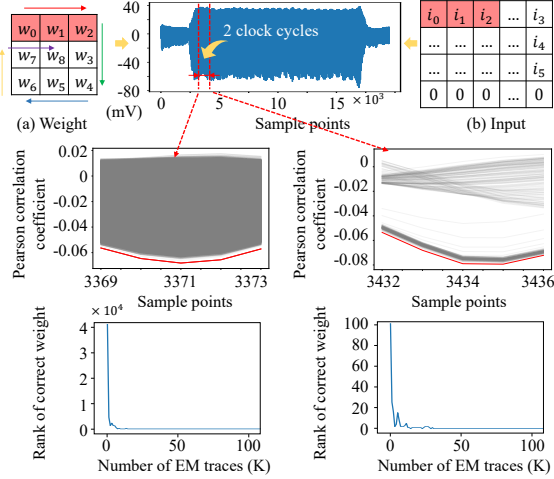


Figure 11. Demonstration of the parameter extract procedure. Note that rank at 0 means perfect recovery.

weights to recover. With weights on the outer ring recovered, we will move on to the second ring, where there is only one weight in the middle for a 3×3 kernel.

7.2.3. Extending the Attack to the Second Layer. After recovering the parameters of the first layer, we can set the output of the first layer by controlling the input layer. The output of the first layer feeds the second layer as input. The challenge lies in ensuring that a significant number of values in the output feature map are zeros. Since the output of the first layer passes through the ReLU function before entering the next layer, we can set the input values of the second layer to zero by making the ReLU inputs non-positive.

We employ the Z3 SMT solver [56] and impose constraints on the input of the first layer, requiring them to be non-negative 8-bit integers. We extract a 2D-convolution kernel from real-world DNN models, allowing the input to convolve with the kernel, and set constraints on the output (e.g., each column has non-positive values, except for the first row, which contains three randomly generated positive values). Furthermore, we define a timeout of one minute and let the solver find solutions. For a 3×3 kernel, we can identify 100K unique inputs that satisfy the constraints in 24 hours, using Z3 V4.12 on a workstation with an Intel E5-2680 v4 CPU. We launch 28 solvers in parallel using multiprocessing. We then repeat the attack on the second layer using these inputs.

7.2.4. Caveats and Future Work.

Uncertainties of the SMT Solver: We empirically tested 10 kernels from real-world models, and the solver was able to find inputs that met the constraints each time. However, we have not proven theoretically that suitable inputs always exist for every possible kernel. Also, the solver does not provide an estimation of the required time as a function of the number of constraints, making it difficult to determine the time needed to find suitable inputs for deeper layers. We plan to address these challenges in future work.

High Cost to Extend to More Complex Cases: With the HD EM model employed for CEMA, we start with discovering the first two weights together. The problem with extending the attack to more than one input or output channel is that many more secret weights are involved in the leakage. For instance, in the case of two output channels, $HD(i_0 w_{0,n}, i_0 w_{0,n} + i_1 w_{1,n})$ and $HD(i_0 w_{0,n+1}, i_0 w_{0,n+1} + i_1 w_{1,n+1})$ occur simultaneously. We cannot focus on one hamming distance and consider the other one as noise, as we do in ciphers because they both depend on the same input values i_0 and i_1 . Therefore, these two HDs are highly correlated. We would need to solve the two HDs, i.e., four unknown weights simultaneously. The complexity of the task is 2^{32} , which is still considered solvable. However, the base case with the complexity 2^{16} takes two hours. Thus, although extending the attack to a more complex layer is feasible, it exceeds our current computation capacity. We examine all the cycles of the DPU using the simulator and find that all of them suffer from this problem. For future work, it might be possible to use a high-precision EM probe to focus on the target arithmetic operations.

8. Discussion and Conclusion

Responsibly Disclosure: Although providing a portal for vulnerability reporting [57], AMD-Xilinx does not have an entry in the form for side-channel vulnerabilities. We reported under the category of *insufficient encryption*, indicating the available DSP configuration codes lead to the reverse engineering of the DPU soft IP. They responded that the encryption was complete and the leaked configurations were “tie-offs to VCC and GND”. If such configuration is not encoded differently than the tie-offs, i.e., hidden in a similar way as the CARRY8’s configuration, it serves as a starting point for our reverse engineering.

In this work, we demonstrate the first successful reverse-engineering attack against a commercial state-of-the-art encrypted FPGA DNN accelerator IP. Our attack is solely built on public information and side-channel analysis, presenting a realistic threat easily exploited by adversaries. We then build a cycle-accurate simulator to reveal the operation flow of any workload deployed on a DPU. We further propose two model extraction attacks based on EM traces and simulation results, targeting the model architecture and parameters respectively.

The set of attacks would not be possible without the following vulnerabilities. First, the Vivado software that synthesizes and compiles the DPU IP explicitly allows the user to view the hierarchy and the schematic while synthesizing an encrypted IP. Moreover, it also reveals critical configurations of the DSP slices in the schematic directly. We consider these a huge confidentiality concern, as the schematic and configuration are leveraged to recover the core hardware structure and its functionality. Second, the DPU design only relies on encryption to protect the IP core, i.e., without any power/EM side-channel countermeasures to prevent IP reverse-engineering attacks. For defense, the designer could add scheduling obfuscation [58], to make it harder to perform side-channel analysis. Also, they could

encrypt the elf model that stores the binary neural network model to prevent any modifications to the deployed model, a necessary step for reverse engineering.

Acknowledgments

We would like to express our profound gratitude to the anonymous shepherd and reviewers, whose insightful feedback greatly enriched the quality of our work. This work was supported in part by the U.S. National Science Foundation through the following grants: CNS-2212010, CNS-1916762, SaTC-1929300, and CNS-2153690.

References

- [1] T. Yu, X. Li, Y. Cai, M. Sun, and P. Li, “S2mlpv2: Improved spatial-shift mlp architecture for vision,” *arXiv preprint arXiv:2108.01072*, 2021.
- [2] “Cloud Tensor Processing Units (TPUs),” Apr. 2023. [Online]. Available: <https://cloud.google.com/tpu/docs/tpus>
- [3] Xilinx, “Vitis-AI,” Nov. 2022, [Online; accessed 28. Nov. 2022]. [Online]. Available: https://github.com/Xilinx/Vitis-AI/tree/master/model_zoo
- [4] jcjohnson, “cnn-benchmarks,” Nov. 2022, [Online; accessed 28. Nov. 2022]. [Online]. Available: <https://github.com/jcjohnson/cnn-benchmarks>
- [5] V. Kathail, “Xilinx vitis unified software platform,” in *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2020, pp. 173–174.
- [6] Xilinx, “Zynq dpu v3.2,” https://www.xilinx.com/support/documentation/ip_documentation/dpu/v3_2/pg338-dpu.pdf.
- [7] Y. Zhang, R. Yasaei, H. Chen, Z. Li, and M. A. Al Faruque, “Stealing neural network structure through remote fpga side-channel analysis,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4377–4388, 2021.
- [8] V. Meyers, D. Gnad, and M. Tahoori, “Reverse engineering neural network folding with remote fpga power analysis,” in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2022, pp. 1–10.
- [9] S. Tian, S. Moini, A. Wolnikowski, D. Holcomb, R. Tessier, and J. Szefer, “Remote power attacks on the versatile tensor accelerator in multi-tenant fpgas,” in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2021, pp. 242–246.
- [10] K. Yoshida, M. Shiozaki, S. Okura, T. Kubota, and T. Fujino, “Model reverse-engineering attack against systolic-array-based dnn accelerator using correlation power analysis,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 104, no. 1, pp. 152–161, 2021.
- [11] G. Li, M. Tiwari, and M. Orshansky, “Power-based attacks on spatial dnn accelerators,” *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 18, no. 3, pp. 1–18, 2022.
- [12] S. Moini, S. Tian, D. Holcomb, J. Szefer, and R. Tessier, “Remote power side-channel attacks on bnn accelerators in fpgas,” in *2021 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2021, pp. 1639–1644.
- [13] “Automotive,” Oct. 2022, [Online; accessed 9. Oct. 2022]. [Online]. Available: <https://www.xilinx.com/applications/automotive.html>
- [14] M. Chiappetta, “Subaru Taps Xilinx For Its New EyeSight Vision-Based Advanced Driver-Assistance System,” *Forbes*, Aug. 2020. [Online]. Available: <https://www.forbes.com/sites/marcochiappetta/2020/08/19/subaru-taps-xilinx-for-its-new-eyesight-vision-based-advanced-driver-assistance-system/>
- [15] splineai cloud, “COVID-XS,” Oct. 2022, [Online; accessed 9. Oct. 2022]. [Online]. Available: <https://github.com/splineai-cloud/COVID-XS>
- [16] “Using IP Encryption in Vivado Design Suite,” Oct. 2022, [Online; accessed 10. Oct. 2022]. [Online]. Available: <https://www.xilinx.com/video/hardware/using-ip-encryption-vivado-design-suite.html>
- [17] Xilinx, “Vitis ai model zoo,” https://github.com/Xilinx/Vitis-AI/tree/master/model_zoo, (Accessed on 10/09/2022).
- [18] A. S. Foundation. Vta: Deep learning accelerator stack. <https://tvm.apache.org/docs/vta/index.html>.
- [19] L. Liu and S. Brown, “Leveraging fine-grained structured sparsity for cnn inference on systolic array architectures,” in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 301–305.
- [20] Y. Yu, T. Zhao, K. Wang, and L. He, “Light-opu: An fpga-based overlay processor for lightweight convolutional neural networks,” in *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2020, pp. 122–132.
- [21] S. Yan, Z. Liu, Y. Wang, C. Zeng, Q. Liu, B. Cheng, and R. C. Cheung, “An fpga-based mobilenet accelerator considering network structure characteristics,” in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 17–23.
- [22] T. Moreau, T. Chen, Z. Jiang, L. Ceze, C. Guestrin, and A. Krishnamurthy, “Vta: an open hardware-software stack for deep learning,” *arXiv preprint arXiv:1807.04188*, 2018.
- [23] “Automotive Solutions,” Apr. 2023. [Online]. Available: <https://www.amd.com/en/solutions/automotive.html>
- [24] M. Chiappetta, “Subaru Taps Xilinx For Its New EyeSight Vision-Based Advanced Driver-Assistance System,” Aug. 2020. [Online]. Available: <https://www.forbes.com/sites/marcochiappetta/2020/08/19/subaru-taps-xilinx-for-its-new-eyesight-vision-based-advanced-driver-assistance-system/?sh=2336702c71f5>
- [25] W. G. Wong, “Xilinx SoC FPGA Powers Baidu’s Apollo Driverless Platform,” Jan. 2020. [Online]. Available: <https://www.electronicdesign.com/markets/automotive/article/21119589/xilinx-soc-fpga-powers-baidus-apollo-driverless-platform>
- [26] “Can 34-Year-Old Chipmaker Xilinx Challenge the Big Players in AI? | Synced,” Nov. 2018. [Online]. Available: <https://syncedreview.com/2018/11/23/can-34-year-old-chipmaker-xilinx-challenge-the-big-players-in-ai>
- [27] Xilinx, “Vitis-AI,” Apr. 2023. [Online]. Available: <https://github.com/Xilinx/Vitis-AI/tree/1.2/AI-Model-Zoo#performance-on-zcu102-0432055-05>
- [28] M. Baharani, U. Sunil, K. Manohar, S. Furgurson, and H. Tabkhi, “Deepdive: An integrative algorithm/architecture co-design for deep separable convolutional neural networks,” in *Proceedings of the 2021 on Great Lakes Symposium on VLSI*, 2021, pp. 247–252.
- [29] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [31] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.
- [32] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.

- [33] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [34] Xilinx. Dpu for convolutional neural network. <https://www.xilinx.com/products/intellectual-property/dpu.html>.
- [35] IEEE, "Ieee recommended practice for encryption and management of electronic design intellectual property (ip)," *IEEE Std 1735-2014 (Incorporates IEEE Std 1735-2014/Cor 1-2015)*, pp. 1–90, 2015.
- [36] Xilinx. Creating and packaging custom ip (ug1118). <https://docs.xilinx.com/r/en-US/ug1118-vivado-creating-packaging-custom-ip/Vivado-Design-Suite-User-Guide-Creating-and-Packaging-Custom-IP>.
- [37] L. Wei, B. Luo, Y. Li, Y. Liu, and Q. Xu, "I know what you see: Power side-channel attack on convolutional neural network accelerators," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 393–406.
- [38] Y. Zhang, R. Yasaei, H. Chen, Z. Li, and M. A. A. Faruque, "Stealing neural network structure through remote fpga side-channel analysis," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4377–4388, 2021.
- [39] V. Meyers, D. Gnad, and M. Tahoori, "Reverse engineering neural network folding with remote fpga power analysis," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–10.
- [40] S. Tian, S. Moini, A. Wolnikowski, D. Holcomb, R. Tessier, and J. Szefer, "Remote power attacks on the versatile tensor accelerator in multi-tenant fpgas," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2021, pp. 242–246.
- [41] Digilent. Ultra96-v2 board. <https://www.avnet.com/wps/portal/us/products/new-product-introductions/npi/aes-ultra96-v2/>.
- [42] Xilinx. Pynq. <http://www.pynq.io/>.
- [43] —. Dpu on pynq. <https://github.com/Xilinx/DPU-PYNQ>.
- [44] Aaronia, "Aaronia pbs2 e & h near field probe set sniffer dc to 6ghz with emc preamplifier," <https://instrumentcenter.eu/products/emc-products/probes/aaronia-pbs2-e-h-near-field-probe-set-sniffer-dc-to-6ghz-with-emc-preamplifier>, (Accessed on 06/06/2021).
- [45] Teledyne, "Teledyne lecroy - oscilloscope," <https://teledynelecroy.com/oscilloscope/>, (Accessed on 06/08/2021).
- [46] G. Becker, J. Cooper, E. De Mulder, G. Goodwill, J. Jaffe, G. Kenworthy *et al.*, "Test vector leakage assessment (tvla) derived test requirements (dtr) with aes," in *International Cryptographic Module Conference*, 2013.
- [47] "Issue 14: Accelerated Verification of Block-Based FPGA Designs | Blue Pearl Software Inc." Jan. 2022. [Online]. Available: <https://www.bluepearlsoftware.com/issue-14-accelerated-verification-of-block-based-fpga-designs>
- [48] Xilinx, "72319 - ip encryption – why am i seeing module names inside encrypted ips as „hidden?“ in vivado?" https://support.xilinx.com/s/article/72319?language=en_US, (Accessed on 07/24/2022).
- [49] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "Hotflip: White-box adversarial examples for text classification," *arXiv preprint arXiv:1712.06751*, 2017.
- [50] K. Leino and M. Fredrikson, "Stolen memories: Leveraging model memorization for calibrated {White-Box} membership inference," in *29th USENIX security symposium (USENIX Security 20)*, 2020, pp. 1605–1622.
- [51] G. Haas and A. Aysu, "Apple vs. ema: electromagnetic side channel attacks on apple corecrypto," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 247–252.
- [52] L. Batina, S. Bhasin, D. Jap, and S. Picek, "{CSI}{NN}: Reverse engineering of neural network architectures through electromagnetic side channel," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 515–532.
- [53] K. Yoshida, T. Kubota, S. Okura, M. Shiozaki, and T. Fujino, "Model reverse-engineering attack using correlation power analysis against systolic array based neural network accelerator," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2020, pp. 1–5.
- [54] S. He, W. Wu, Y. Li, L. Zhou, L. Fang, and Z. Liu, "Recovering the weights of convolutional neural network via chosen pixel horizontal power analysis," in *International Conference on Wireless Algorithms, Systems, and Applications*. Springer, 2022, pp. 93–104.
- [55] V. Yli-Mäyry, A. Ito, N. Homma, S. Bhasin, and D. Jap, "Extraction of binarized neural network architecture and secret parameters using side-channel information," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [56] L. De Moura and N. Björner, "Z3: An efficient smt solver," in *Tools and Algorithms for the Construction and Analysis of Systems: 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29–April 6, 2008. Proceedings 14*. Springer, 2008, pp. 337–340.
- [57] "Embedded Web Form," Dec. 2022, [Online; accessed 2. Dec. 2022]. [Online]. Available: <https://www.xilinx.com/about/feedback/embedded-web-form.html>
- [58] Y. Luo, S. Duan, C. Gongye, Y. Fei, and X. Xu, "Nnresearch: A tensor program scheduling framework against neural network architecture reverse engineering," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–9.
- [59] R. Wu, T. Kim, D. J. Tian, A. Bianchi, and D. Xu, "{DnD}: A {Cross-Architecture} deep neural network decompiler," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2135–2152.
- [60] Z. Liu, Y. Yuan, S. Wang, X. Xie, and L. Ma, "Decompiling x86 deep neural network executables," *arXiv preprint arXiv:2210.01075*, 2022.
- [61] A. Bourenane, "Well Received," Nov. 2021. [Online]. Available: <https://digilent.com/shop/pynq-z1-python-productivity-for-zynq-7000-arm-fpga-soc>

Appendix A. AMD-Xilinx DPU Configuration

TABLE 5. EXPERIMENTAL DPU CONFIGURATION

Architecture Model [6]	B1600
RAM usage (Disable/Enable)	Disable
RAM usage (Low/High)	Low
Channel augmentation (Disable/Enable)	Disable
Depth wise Convolution (Yes/No)	Yes
Average Pooling (Yes/No)	Yes
Activation function (ReLU, ReLU6)	ReLU
DSP48 usage (Low/High)	High
DSP data rate & Registers data rate on Ultra96	
Data transmission clock (Max 185MHz)	160MHz
DSP clock (Max 370MHz)	320MHz

Appendix B. Reverse-Engineer the Binary File

The binary neural network model, i.e., the elf file generated by the Vitis compiler, contains labeled binary blobs for weights in quantized form with an unknown sequence. To control the elf file, we must decode the sequence. Previous techniques [59], [60] that reverse-engineered DNN binary

do not apply here, because DPU has proprietary ISA and is exclusively designed for highly optimized quantized DNNs. We first reverse engineer the quantization scheme, then develop an automated algorithm to find the weight coordinates. The weights are stored in a 4D matrix represented as (OC, IC, K_w, K_h) . Last, we derive an algorithm for ordering weights in the binary model, shown in Algorithm 2.

Algorithm 2 Sequencing the weights in the binary file

```

1: for  $oc = 0$  to  $OC$  do
2:   for  $k_w = 0$  to  $K_w - 1$  do
3:     for  $k_h = 0$  to  $K_h - 1$  do
4:       for  $ic = 0$  to  $IC$  do
5:         binaryIndex.value  $\leftarrow (oc, ic, k_w, k_h).value$ 
6:         binaryIndex  $\leftarrow$  binaryIndex + 1

```

Appendix C.

TVM-VTA Reverse Engineering

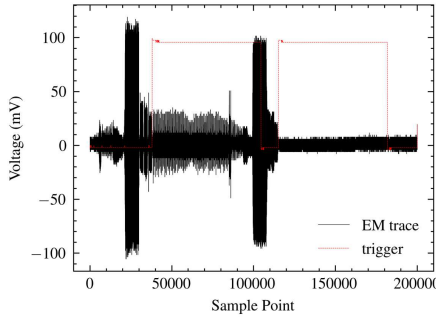


Figure 12. A sample EM trace with write-back signals.

C.1. Software Scheduling Reverse Engineering

To demonstrate the generality of our reverse engineering approach, we apply the knowledge from DPU to reverse-engineer TVM-VTA, another representative instruction-flow-based FPGA DNN accelerator.

C.1.1. EM Trace Collection and Segmentation. For the TVM-VTA, we select the store signal as the trigger, as illustrated in Figure 12. The start of each store signal provides an ideal timing reference for both trace segmentation and operation alignment. We select the PYNQ-Z1 development board [61] as the target experimental platform and install the PYNQ v2.6 operating system driven by the PetaLinux kernel [42], [43]. We deploy DNN models supported by the TVM on the target platform. The VTA configuration specifies the number of computing resources. We choose the default configuration with $PP = 1$, $ICP = 16$ and $OCP = 16$.

C.1.2. Intra-layer Workload Segmentation.

Step 1: We observe that both the input height and width affect the number of segmentations. Our observations are formulated below:

$$N = \lceil \frac{H}{H_b} \rceil \lceil \frac{W}{W_b} \rceil \quad (5)$$

This means that the input is segmented into rectangle tiles, where $H_b = 7$, $W_b = 14$.

Step 2: Similar to DPU, we have the following observations for VTA:

$$N = \lceil \frac{IC}{ICP} \rceil \lceil \frac{OC}{OCP} \rceil \quad (6)$$

This result indicates that the hardware structure can only handle the workload at once when both IC and OC do not exceed ICP and OCP . Otherwise, the hardware is reused in an unrolling fashion.

Step 3: We formulate the relationship between the length of the segment and the parameters below, where Δt is the duration of the segment, s is the stride size, f is the operating frequency of the accelerator, and C is the number of clock cycles.

$$\frac{\Delta t}{f} = C \approx \frac{(W - K_w + 1)(H - K_h + 1)}{s} K_w K_h \quad (7)$$

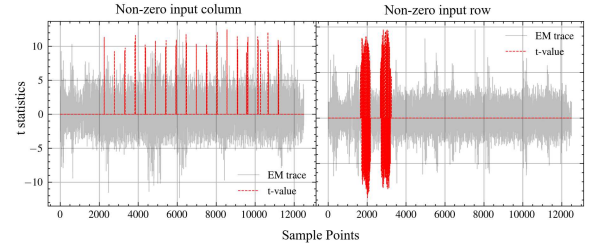


Figure 13. TVM-VTA i -test to determine the order of the input

C.1.3. Intra-segment Scheduling.

Step 1: As demonstrated in Figure 13, the input order is row-major.

Step 2: VTA's kernel execution order is row-major.

Step 3: VTA's input channels start at the same cycle. Output channels 0...15 each start one cycle later than the previous one. Output channel 0 starts at the same time as input channel 0.

C.2. Hardware Hierarchy Reverse Engineering

Step 1: We identify two major blocks with 256 and 16 DSP slices, respectively, as shown in Figure 14. The 256-DSP block is consist of 16×16 tiles and each tile has one DSP slice.

Step 2: Using template matching, we discover that there are two types of tiles. One performs multiplication only, the MUL tile. The other performs accumulation only, the ACC tile. The outputs of each column of the MUL tiles go to an adder tree and reduce to a scalar in a corresponding ACC tile.

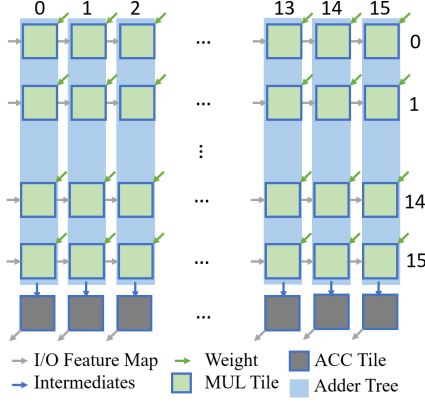


Figure 14. The core hardware structure of the TVM-VTA.

Step 3: For a MUL tile, we need to identify the flows of inputs and weights. We notice that the horizontal data flow goes to the next column as well. Hence, we infer that the 128-bit input from the left shown in Figure 14 is the input feature map. The other non-reused input is the weight.

In summary, following these three steps, we establish that each column is identical. It processes inputs entering Column 0, carries out multiplications in MUL tiles, and releases outputs through the adder tree to the ACC tiles. Combining with the scheduling information we can easily see that each row takes one input channel of the input and each column handles one output channel.

C.3. Result and Evaluation

The number of cycles of a TVM-VTA is as follows:

$$C = \frac{(W - K_w + 1)(H - K_h + 1)}{s} K_w K_h + 15 \quad (8)$$

where 15 is the number of pipeline stages.

Each column of the MUL tiles has the same functionality, hence we only need to demonstrate the scheduling of one column, which we demonstrate in Algorithm 3. We build a simulator according to the algorithm and verify that the cycles and outputs simulated match those of the real hardware.

Appendix D.

sDPU Evaluation Results

We present additional unique Conv2D and Dense workload configurations summarized from the Model Zoo, which we use to construct an evaluation set. By comparing our simulation results with the actual execution time measured using EM measurements on DPU, we confirm the cycle accuracy of our sDPU.

Algorithm 3 Scheduling the weight and input for column 0

Require: Feature, Filter

```

1:  $\triangleright (Feature, 3D \text{ array})(IC, W, H)$ : input feature map  $\triangleleft$ 
2:  $\triangleright (Filter, 4D \text{ array})(OC, IC, K_w, K_h)$ : filters,  $OC \leftarrow 1$   $\triangleleft$ 
Ensure: Stream
3:  $\triangleright (Stream 2D \text{ array})(cycles, nDSP)$  the data stream that contains
   weight and input feature for each DSP at each cycle  $nDSP=nRow$   $\triangleleft$ 
    $\triangleright$  there are 16 MUL tiles
4:  $nRow \leftarrow 16$   $\triangleright$  there are 16 MUL tiles
5:  $IC \leftarrow nRow$   $\triangleright$  Each row can support one input channel
6:  $nDSP \leftarrow nRow$   $\triangleright$  there are one DSP slice in each MUL tile
7:  $\triangleright$  Number of cycles to complete one row of a kernel  $\triangleleft$ 
8:  $CyclePerKernelRow \leftarrow K_w$ 
9:  $\triangleright$  Number of cycles to complete all rows of a kernel  $\triangleleft$ 
10:  $CycleAllKernelRow \leftarrow CyclePerKernelRow \cdot K_h$ 
11:  $\triangleright$  number of cycles to process this feature map  $\triangleleft$ 
12:  $cycles = (W - K_w + 1)((H - K_h + 1) \cdot K_w \cdot K_h + (nDSP - 1))$ 
13: for  $c=0$  to  $cycles$  do
14:   for  $row=0$  to  $nRow$  do
15:      $k_w \leftarrow c \bmod K_w$ 
16:      $k_h \leftarrow \lfloor c / K_h \rfloor \bmod k_h$ 
17:      $w \leftarrow k_w$ 
18:      $h \leftarrow \lfloor c / CycleAllKernelRow \rfloor + k_h$ 
19:      $Stream[c][row].in \leftarrow Feature[row, w, h]$ 
20:      $Stream[c][row].w \leftarrow Filter[oc, row, k_w, k_h]$ 

```

TABLE 6. DENSE WORKLOAD BENCHMARK EVALUATION RESULTS.

Dense Workload					EM trace Measured		sDPU
IC	OC	K_h/w	IF_h/w	OF_h/w	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
512	1000	1	1	1	211250	66016	67600
576	1000	1	1	1	235625	73633	75400
768	3072	1	1	1	963462	301082	308308
768	768	1	1	1	240865	75271	77077
768	2	1	1	1	3128	978	1001
768	1000	1	1	1	312812	97754	100100
1024	1000	1	1	1	418437	130762	133900
1024	4096	1	1	1	1715593	536123	548990
1024	1024	1	1	1	430990	134685	137917
1024	2	1	1	1	4184	1308	1339
1280	1000	1	1	1	520000	162500	166400
1408	1000	1	1	1	572812	179004	183300
1536	1000	1	1	1	625625	195508	200200
1664	1000	1	1	1	678437	212012	217100
1792	1000	1	1	1	731250	228516	234000
1920	1000	1	1	1	780000	243750	249600
2048	1000	1	1	1	832812	260254	266500
2304	1000	1	1	1	938437	293262	300300
2560	1000	1	1	1	1040000	325000	332800
3072	768	1	1	1	963462	301082	308308
4096	1024	1	1	1	1715593	536123	548990

TABLE 7. CONV2D WORKLOAD BENCHMARK EVALUATION (PART1).

Conv2D Workload					EM trace Measured		sDPU
IC	OC	K_h/w	IF_h/w	OF_h/w	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
1	64	3	128	128	93450	29204	29904
1	64	3	352	352	6074337	1898231	1943788
2	12	3	16	16	3287	1028	1052
2	64	1	64	128	24325	7602	7784
2	64	1	128	128	93450	29204	29904
2	1024	1	8	8	8690	2716	2781
3	16	3	224	112	701225	219133	224392
3	32	3	224	112	1402450	438266	448784
3	32	3	256	256	1833200	572875	586624
3	32	3	480	480	6461250	2019141	2067600
3	64	7	224	112	13092187	4091309	4189500
3	64	3	224	112	2454287	766965	785372
3	64	3	224	224	2454287	766965	785372
3	64	3	800	800	31445312	9826660	10062500
8	16	3	512	1024	3676400	1148875	1176448
12	64	1	16	16	3762	1176	1204
16	3	3	512	512	3676400	1148875	1176448
16	3	5	512	256	10164400	3176375	3252608
16	16	3	112	112	348425	108883	111496

TABLE 8. CONV2D WORKLOAD BENCHMARK EVALUATION (PART2).

Conv2D Workload					EM trace Mesured		sDPU
IC	OC	$K_{h/w}$	$IF_{h/w}$	$OF_{h/w}$	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
16	32	3	1024	512	58902400	18407000	18848768
16	64	1	112	112	143937	44981	46060
16	96	1	112	112	205625	64258	65800
24	72	1	56	56	64575	20180	20664
24	144	1	56	56	121078	37837	38745
32	1	1	480	480	728250	227579	233040
32	3	3	299	149	2544575	795180	814264
32	3	7	256	256	9804400	3063875	3137408
32	16	1	112	112	82250	25704	26320
32	32	3	112	112	1393700	435532	445984
32	32	3	149	147	2524150	788797	807728
32	32	3	256	128	7332800	2291500	2346496
32	32	3	128	64	1823200	569750	583424
32	32	3	256	256	7332800	2291500	2346496
32	32	3	128	128	1823200	569750	583424
32	32	3	64	64	450800	140875	144256
32	32	3	32	32	110200	34438	35264
32	32	3	16	16	26300	8219	8416
32	32	3	8	8	5950	1860	1904
32	32	3	4	4	2350	735	752
32	64	1	112	112	287875	89961	92120
32	64	3	147	73	4357412	1361692	1394372
32	64	3	128	256	3190600	997063	1020992
32	64	3	512	256	51469600	16084250	16470272
32	64	3	128	128	3190600	997063	1020992
32	64	3	240	240	11274375	3523243	3607800
32	128	1	56	56	139912	43723	44772
32	192	1	28	28	67000	20938	21440
32	256	1	256	256	5439200	1699750	1740544
32	256	3	256	256	47663200	14894750	15252224
40	120	1	28	28	40200	12563	12864
48	192	1	28	28	83750	26172	26800
48	256	1	28	28	108875	34024	34840

TABLE 9. CONV2D WORKLOAD BENCHMARK EVALUATION (PART3).

Conv2D Workload					EM trace Mesured		sDPU
IC	OC	$K_{h/w}$	$IF_{h/w}$	$OF_{h/w}$	Latency (ns)	Clock Cycle (cc)	Clock Cycle (cc)
64	1	1	128	128	93450	29204	29904
64	3	7	224	112	13092187	4091309	4189500
64	3	3	128	64	797650	249266	255248
64	3	3	256	256	3208100	1002532	1026592
64	3	3	224	224	2454287	766965	785372
64	16	1	112	112	143937	44981	46060
64	16	1	512	512	2898000	905625	927360
64	16	1	256	256	732200	228813	234304
64	24	1	56	56	56503	17658	18081
64	32	3	120	240	2802187	875684	896700
64	64	1	56	56	131840	41200	42189
64	64	3	56	56	1053653	329267	337169
64	64	1	112	112	503781	157432	161210
64	64	1	512	512	10143000	3169688	3245760
64	64	3	512	256	90071800	28147438	28822976
64	64	3	5	5	9953	3111	3185
64	64	3	3	3	4440	1388	1421
64	64	3	64	32	1380575	431430	441784
64	64	3	32	32	337487	105465	107996
64	64	3	16	16	80543	25170	25774
64	64	3	8	8	18221	5695	5831
64	64	3	4	4	7196	2249	2303
64	64	3	128	64	5583550	1744860	1786736
64	64	3	32	16	337487	105465	107996
64	64	3	64	64	1380575	431430	441784
64	64	3	32	32	337487	105465	107996
64	64	3	16	16	80543	25170	25774
64	64	3	8	8	18221	5695	5831
64	64	3	4	4	7196	2249	2303
64	64	3	224	224	17180012	5368754	5497604
64	64	3	176	176	10587981	3308745	3388154
64	64	3	88	88	2625940	820607	840301
64	64	3	44	44	704681	220213	225498
64	64	3	22	22	170428	53259	54537
64	64	3	11	11	52981	16557	16954
64	64	3	6	6	12709	3972	4067
64	64	1	3	3	2603	814	833
64	64	3	128	128	5583550	1744860	1786736
64	96	1	73	73	343437	107325	109900
64	128	1	56	28	244846	76515	78351
64	128	3	104	104	6828128	2133790	2185001
64	128	3	56	28	1956784	611495	626171
64	128	1	56	56	244846	76515	78351
64	128	1	16	16	24456	7643	7826
64	128	3	32	64	626762	195864	200564
64	128	3	64	128	2563925	801227	820456
64	128	3	256	128	41705300	13032907	13345696
64	128	3	128	256	10369450	3240454	3318224
64	128	3	64	64	2563925	801227	820456
64	128	3	400	400	102019531	31881104	32646250
64	128	3	112	112	7926668	2477084	2536534
64	128	3	120	120	9107109	2845972	2914275
64	192	3	56	56	3010437	940762	963340
64	256	1	56	56	489693	153030	156702
64	256	1	14	14	44362	13864	14196
72	40	1	28	28	26800	8375	8576
72	72	3	28	28	383200	119750	122624
80	64	1	73	73	274750	85860	87920
96	24	1	56	56	80718	25225	25830
96	96	3	56	56	2150312	671973	688100
96	128	3	56	28	2795406	873565	894530
96	192	3	73	35	8056250	2517579	2578000
120	48	1	14	14	14625	4571	4680
120	120	3	14	14	204300	63844	65376
128	3	1	80	80	69468	21709	22230
128	32	1	56	56	139912	43723	44772
128	64	1	104	104	809615	253005	259077
128	64	3	112	56	7926668	2477084	2536534
128	64	1	56	56	244846	76515	78351
128	64	1	256	256	4759300	1487282	1522976
128	64	1	128	128	1214850	379641	388752
128	64	3	64	32	2563925	801227	820456
128	64	3	128	64	10369450	3240454	3318224
128	64	3	256	128	41705300	13032907	13345696
128	64	3	60	120	2400125	750040	768040
128	128	3	28	28	1011887	316215	323804
128	128	1	56	56	454715	142099	145509
128	128	1	256	256	8838700	2762094	2828384
128	128	3	256	128	77452700	24203969	24784864
128	128	3	128	64	19257550	6017985	6162416
128	128	3	128	128	19257550	6017985	6162416
128	128	3	64	32	4761575	1487993	1523704
128	128	3	32	32	1163987	363746	372476
128	128	3	16	16	277793	86811	88894

Appendix E. Meta-Review

E.1. Summary

This paper proposes to reverse engineering hardware DNN accelerator IPs using EM side channels. The pipeline includes SEMA and t-test. The attack has been demonstrated on TVM-VTA and AMD-Xilinx DPU using simple DNN models.

E.2. Scientific Contributions

- Provides a Valuable Step Forward in an Established Field
- Creates a New Tool to Enable Future Science
- Identifies an Impactful Vulnerability

E.3. Reasons for Acceptance

- 1) Both SCA using EM and DNN model stealing directions have been established as important fields. This paper marries these two fields and provides values to both areas.
- 2) The corresponding attack pipeline and the simulator create new tools to benefit both research areas further.

E.4. Noteworthy Concerns

Attacks are limited to PoC for simple DNN models rather than real-world IPs.